

## Problem and Motivation

GPT-2 is a decoder-only transformer language model that processes sequences of tokens through a stack of identical layers, each comprising a multi-head self-attention sublayer followed by a position-wise feed-forward network (MLP). I propose to implement an accelerator for the training process of the GPT-2 LLM. The training algorithm includes both forward and backwards passes, which increases the number of matrix multiplications required per step from one to three.

Profiling the algorithm determined that the three multiplication operations combine to represent 86% of computation time for the training algorithm. My accelerator therefore primarily targets matrix multiplication. Targeting exclusively matrix multiplication results in a theoretical speedup of 7.14x limit with infinite speedup of the targeted kernel. Targeting specifically matrix multiplication allows for construction of a much smaller accelerator than would be necessary to speed up the entire design, as it allows data to be streamed in and out, rather than having to hold entire matrices on chip in SRAM if portions of the algorithm like normalization were included as acceleration targets.

The three types of matrix multiplication exhibit identical arithmetic structure. They differ only in which operand is held stationary and which is streamed, which makes a weight-stationary systolic array a natural architectural fit. A set of weights can be loaded, and 256 activation rows streamed through it, allowing excellent natural tiling of the multiplication. The large size of the matrices ( $[256 \times 768] \times [768 \times 3072]$  for forward matrix multiplication) result in high arithmetic intensity, which in turn implies a high theoretical throughput speedup without increasing the memory bandwidth.

## Roofline Analysis

The baseline system tested against is my laptop, which has a maximum compute throughput of 204.8 GFLOPS and a maximum memory bandwidth of 38.4 GB/s, for a ridge point of 5.33 FLOP/B. This places the matrix multiplication portion of the original algorithm, with an arithmetic intensity of 33.5 FLOP/B, deeply in the compute-bound portion of the graph (Figure 1). The primary goal of my accelerator was therefore to provide sufficient compute bandwidth to exploit that high arithmetic intensity, shifting the bottleneck from compute capacity to memory bandwidth, where throughput scales directly with the AXI bus and is ultimately limited by DRAM bandwidth.

With a  $32 \times 32$  systolic array as its compute core, the maximum throughput of my accelerator was determined to be 1.24 TFLOPS at 606 MHz, while the data bandwidth of the 512-bit AXI-Stream bus was determined to be 38.75 GB/s. This places the ridge point of my accelerator at 32.3 FLOP/B, much closer to the arithmetic intensity of the kernel.

Interestingly, the matrix multiplication implementation on the accelerator has worse arithmetic intensity than the original CPU implementation. This is due to the design decision to keep on-chip SRAM buffers as small as possible, which results in better cache behavior on the CPU than on the accelerator. This in turn requires more data re-streaming than is present in the CPU implementation. For forward matrix multiplication and dout matrix multiplication, the arithmetic intensity is 25.8 FLOP/B on the accelerator. For the dweight matrix multiplication it is further reduced to 16 FLOP/B. This reduction for dweight is due to this multiplication operation requiring weight swaps to occur every 32 activation columns computed, rather than every 256 activation columns for forward and dout multiplication. Each

multiplication is the same size, so the average arithmetic intensity of all accelerated kernels is 22.5 FLOP/B. This moves the implementation into the memory-bound portion of the roofline, though at a much higher absolute throughput than the original.

## Dataflow and Architecture

My design follows the traditional methodology of a compute intensive kernel hardware accelerator, consisting of an input bus, which will be discussed in a future section, an SRAM buffer for input values, a compute core, output SRAM, and an interface to return data to the host. My design includes one component not in the traditional design, namely a 20 stage pipeline that implements GELU operations on the returning data before sending it back to the host. Figure 2 is a block diagram of my design.

The interface consists of a 512 bit AXI stream bus, which streams into one of two buffers. The weight buffer is 4KB SRAM split into two banks. Each bank is 2KB, enough space to hold the weights to completely fill the systolic array. The two banks allow double buffering, where weight data can be streamed into one buffer while the contents of the other buffer are loaded into the systolic array, thereby avoiding sequential weight streaming and loading. The remaining buffer is the 2KB activation buffer, which holds activations to be fed into the systolic array. It is necessary because activations must be fed into the array in a staggered fashion, which requires the first activation column to be present in memory for the full 32 cycles of its entry into the systolic array. By the time the last element of this column is loaded, the first element of the 32nd column is also loaded, which dictates the required size of the buffer. The 32 columns of 32, 2 byte elements total to 2KB, the size of the activation buffer.

The compute core of my design is a 32x32 weight stationary systolic array. The decision to use a weight stationary design is based on the limited data bandwidth of the design. The 512 bit AXI bus used to transfer data to the accelerator has roughly equivalent bandwidth to the DRAM that backs it. This means that there is no way to lift the accelerator's current bandwidth limit. Shifting from a weight stationary to output stationary design would require streaming both weights and activations, which would double the required bandwidth of the current design. Existing bandwidth is already fully utilized to stream activations to the systolic array, so an output stationary design would require dropping from a 32x32 systolic array down to a 16x16 implementation, a 4x throughput reduction.

This design targets three different matrix multiplication operations, the forward, weight gradient, and input gradient operations. Immediately following the forward multiplication in the algorithm is GELU computation for each of the outputs. Given that the data is already present, I decided to implement the GELU computation in hardware, as it requires no additional data movement, and, since it is pipelined, results in near infinite effective speedup. In software GELU is computed:

$$GELU(x) \approx 0.5 \cdot x \cdot \left( 1 + \tanh \left( \sqrt{\frac{2}{\pi}} \cdot (x + 0.044715 \cdot x^3) \right) \right)$$

A direct implementation of tanh would significantly increase area and latency relative to a polynomial approximation, motivating the use of a Padé approximation. I opted to approximate this equation with the

following Padé [3/2] rational approximation.

$$\tanh(u) \approx u \cdot \frac{1485 + 171 \cdot u^2 + 2 \cdot u^4}{1485 + 666 \cdot u^2 + 26 \cdot u^4}$$

This is implemented as a 20 stage pipeline with every addition and multiplication placed in its own clock cycle. The rationale behind this deep pipeline is that multiplications and addition operations are performed in FP32, rather than BF16 for multiplications in the systolic array. This prevents accuracy losses associated with truncating data each time a multiply occurs in the pipeline. FP32 multiplication takes approximately twice as long as BF16 multiplication, and my clock period is only long enough to hold one BF16 multiplication and one FP32 accumulation. This required me to break all operations up into their own clock cycles to avoid having to extend the clock period.

The final destination for data is the AXI stream output bus, where it is returned to the host for further processing. It should be noted that the aforementioned GELU pipeline is only used for the forward matrix multiplication, and that for the weight gradient and input gradient multiplications it is bypassed and the results of matrix multiplication are sent directly to the output bus.

## Hardware Interface

The accelerator uses a 512-bit AXI-Stream bus. At 606 MHz, it provides 38.7 GB/s of bandwidth, effectively matching DRAM throughput. AXI-Stream was chosen over PCIe for its design simplicity at this bandwidth scale, while lower-throughput protocols (such as SPI or I2C) were fundamentally insufficient.

The decision to maximize bandwidth is based on the roofline analysis from the previous section, which indicated that the accelerated kernel implementation is memory bound rather than compute bound as it was in the CPU implementation. This is due to several factors. First is that compute acceleration lifts the throughput maximum by ~6x, which shifts the ridgepoint to the right by the same amount given the same memory bandwidth. Second is that the accelerator algorithm is slightly less efficient in terms of data reuse than the original design, which shifts its arithmetic intensity to the left. The net result is that optimizing throughput requires maximizing the bandwidth of the interface and ensuring that it is always active.

The activity ratio of the read side of the interface is 98.9%. Several design decisions lead to this near perfect utilization. Both revolve around the decision to double buffer the weight and accumulator SRAM. Weight double buffering allows weight loading into the systolic array and weight streaming from the DRAM to occur concurrently during the drain phase of the previous tile's computation. The 32 drain cycles are therefore productively utilized by the bus, and the weights are in place in the systolic array, ready to start processing data from the new tile immediately. SRAM double buffering allows for data processing to continue while fully computed output values are streamed back to the host. If this double buffering were not implemented, computation would have to stall during these cycles, which would require pausing data transfer over the bus as well.

## Precision and Data Format

The original CPU implementation of my algorithm uses FP32 for all floating point operations. In order to reduce the pressure on the AXI bus I opted to use BF16 floating point for data transfers to the accelerator, as well as for all multiplication within the design. Accumulation is still performed using FP32. The decision to use this data format was based on comparisons of output weights after a single training step to the outputs from the original CPU design, with the goal of keeping the mean relative distance between the two at less than 1%. At the time of testing, only the forward pass matrix multiplication was implemented, so a more stringent requirement of less than .3% mean relative distance was required, with the assumption that the backwards pass would result in at least as much deviation as the forward pass. The results of that analysis are as follows:

| Configuration                                      | Mean Rel. $\Delta$ | Cosine Similarity |
|----------------------------------------------------|--------------------|-------------------|
| <b>BF16 <math>\times</math> FP32 (implemented)</b> | <b>0.0183%</b>     | <b>0.99999999</b> |
| INT16 $\times$ FP32                                | 0.0173%            | 0.99999999        |
| FP8 $\times$ FP32                                  | 0.5733%            | 0.99999966        |
| INT8 $\times$ FP32                                 | 0.5710%            | 0.99999966        |
| BF16 $\times$ BF16 (no FP32 accum)                 | 0.8135%            | 0.9999995         |
| Final hardware                                     | 0.8011%            | 0.99999951        |

The best performing configuration for the forward pass was INT16  $\times$  FP32, but this option was discarded as its strong performance was only due to low dynamic range of the values in the forward pass. This is not the case for the backward pass multiplications, where it was expected to produce a sharp increase in mean relative distance. FP8 and INT8 were both disqualified as they produced a mean relative difference of greater than .5%, which would ensure that the net mean relative difference was greater than 1% once backwards pass multiplication was implemented. Using BF16 accumulation was also rejected for the same reason.

The GELU pipeline also uses FP32 multiplication and FP32 addition. This decision requires a 20 stage GELU pipeline, but minimizes quantization error. The motivation behind this decision is that extending the pipeline does not result in a performance hit, so the only cost is increasing chip size and energy consumption, neither of which were priorities for optimization in the design.

The final hardware measurement was included in the table, but should not be compared to the other values, as the conditions under which the data was collected was different than the previous table rows. These data were collected after all matrix multiplication operations and the forward pass GELU had been moved to the accelerator, and indicates that the final implementation achieves the original goal of keeping the mean relative difference between the output weights of a single pass of the accelerator and a single pass of the original software implementation less than 1%.

## Verification

There was a three stage verification process that was implemented for this design. Each module was tested upon completion to determine if it was functioning correctly. A second round of testing was completed at the top level of the design, to determine if the interconnections were functioning properly,

and to test the top level implementation. Finally, the functional behavior of the hardware was tested by integrating the simulated design directly into the software training loop.

Individual Verilator testbenches were written for each submodule of the design. These included the processing element (PE), the systolic array, the weight and activation SRAMs, the AXI input interface, the AXI readback interface, and the controller. Each test was constructed to isolate the behavior of a single module by driving its inputs directly and checking its outputs against analytically computed expected values. This approach allowed functional errors to be identified and corrected close to the point of introduction, before they could propagate into harder-to-diagnose failures at higher levels of integration.

Following module-level verification, a comprehensive top-level testbench was developed targeting the complete top module. The testbench was compiled and simulated using Verilator. Seventeen tests were executed, progressing from simple functional checks to boundary, stress, and timing-sensitive regression cases:

| Test | Description                                                                                               | Purpose                                                                                                               |
|------|-----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| 1    | $M=4, W=1, \text{act}[m]=m+1 \rightarrow \text{out}[m]=\text{ROWS} \times (m+1)$                          | Basic streaming correctness; verifies per-row scaling                                                                 |
| 2    | Two K-tiles, $W=1, \text{act}=1 \rightarrow \text{out}=2 \times \text{ROWS}$                              | Multi-tile accumulation via ping-pong weight banks                                                                    |
| 3    | $M=16, W=2, \text{act}=1 \rightarrow \text{out}=2 \times \text{ROWS}$                                     | Larger M count; verifies stagger at deeper pipeline fill                                                              |
| 4    | Backward mode ( $\text{mode}=1$ ), $W=1, \text{act}=1 \rightarrow \text{out}=\text{ROWS}$                 | Backward datapath                                                                                                     |
| 5    | $\text{Bias}=0.5, W=1, \text{act}=1 \rightarrow \text{out}=\text{ROWS}+0.5$                               | Uniform bias addition via bias SRAM                                                                                   |
| 6    | GELU only, no bias $\rightarrow \text{GELU}(\text{ROWS})$                                                 | GELU post-processing on positive input                                                                                |
| 7    | $\text{Bias}=0.5 + \text{GELU} \rightarrow \text{GELU}(\text{ROWS}+0.5)$                                  | Combined bias and GELU path                                                                                           |
| 8    | Identity weight matrix $\rightarrow \text{out}[m][c]=c+1$                                                 | Regression for weight-row reversal fix                                                                                |
| 9    | $M=1$ (minimum)                                                                                           | Lower boundary of $M\_count$ ; single-row drain                                                                       |
| 10   | $M=64$ (stress)                                                                                           | Verifies accumulator address sequencing at scale                                                                      |
| 11   | $\text{act}=-1, W=1 \rightarrow \text{out}=-\text{ROWS}$                                                  | Negative activation sign handling in BF16                                                                             |
| 12   | $\text{bias}[c]=c+1 \rightarrow \text{out}[m][c]=\text{ROWS}+c+1$                                         | Per-column bias; each column gets its own bias value                                                                  |
| 13   | $W=-1/\text{ROWS}, \text{act}=1 \rightarrow \text{raw}=-1, \text{GELU}(-1) \approx -0.159$                | GELU correctness on a negative pre-activation value                                                                   |
| 14   | $M=256$ ( $M\_MAX$ )                                                                                      | Full accumulator depth; address counter rollover                                                                      |
| 15   | 4 K-tile accumulation $\rightarrow \text{out}=4 \times \text{ROWS}$                                       | Accumulation correctness beyond two tiles                                                                             |
| 16   | Ping-pong preloading: $W=1$ bank 0, preload $W=2$ bank 1 $\rightarrow \text{out}=3 \times \text{ROWS}$    | Preloading timing; AXI write to idle bank during active computation                                                   |
| 17   | drain-overlap $\text{LOAD\_WT}$ : $W=1$ bank 0, preload $W=2$ bank 1, $\text{preload\_rdy}=1 \rightarrow$ | drain-overlap controller path; verifies $\text{IDLE} \rightarrow \text{STREAM}$ skip when $\text{wt\_preloaded\_r}=1$ |

|  |            |  |
|--|------------|--|
|  | out=3×ROWS |  |
|--|------------|--|

The final and most stringent verification stage embedded the cycle-accurate Verilator model directly into the GPT-2 C training source code. Verilator compiled the `sys_top` RTL into a C++ class (`Vsys_top`), which was instantiated in a hardware abstraction layer (`accel_hal_systolic_vrl.cpp`). The HAL drives the DUT's AXI-Stream slave port, control signals, and readback interface on a cycle-by-cycle basis using the same `tick()` primitive used in the standalone testbenches, but now called from within the training loop rather than from a dedicated testbench. From the perspective of the training code, the Verilator model is indistinguishable from a real hardware device: it receives weight tiles and activation slices through the same HAL API and returns FP32 results through the same readback interface.

Gradient outputs were compared against the original CPU-only reference implementation. While these values were expected to have some deviation from the originals due to precision reduction in the design, significant ( $>1\%$ ) differences in mean relative distance were viewed as erroneous and a sign that there was an error in the implementation.

## Synthesis Results

Synthesis was performed using the Cadence tool chain, specifically Genus 23.10, targeting the ASAP7 7nm RTV standard cells. The rationale behind using ASAP7 rather than SKY130, which is typical for a tapeout, is that I wanted the timing, power, and size values determined in synthesis to better reflect the modern accelerator and GPU space, where cells may be as small as 2nm for cutting edge fabrication. Using 7nm cells puts me in the same order of magnitude as real designs, whereas the results of SKY130 synthesis could be as many as two orders of magnitude off just due to cell size.

Total estimated power is 1.140 W. Sequential logic (registers and flip-flops) accounts for 0.575 W (50.4%), while combinational logic accounts for 0.565 W (49.6%). The total cell area is 533,440.909  $\mu\text{m}^2$ , distributed across three primary structures. The accumulator SRAM accounts for 291,384  $\mu\text{m}^2$  (54.6% of total), implemented as two banks of 144,758  $\mu\text{m}^2$  and 146,626  $\mu\text{m}^2$  respectively. The systolic array occupies 177,943  $\mu\text{m}^2$  (33.4%), with each of the 1,024 PEs contributing 173.8  $\mu\text{m}^2$ . The remaining 12.0% covers the AXI-Stream input controller, readback unit (including 32 GELU pipeline instances), weight SRAMs, and interconnect.

Of note is the high contribution of sequential logic to power and accumulator SRAM to the area. The reason for both is a limitation of ASAP7. ASAP7 does not have SRAM cells, and as such I was forced to synthesize all on chip storage as flip-flops instead. Flip flops consume 10-15x as much power as SRAM and 5-10x as much space. While I do not have a direct breakdown of how much of the sequential logic is required to be implemented as flip flops, the vast majority could be replaced with SRAM cells if they were available, which would conservatively reduce the sequential logic power consumption by a factor of 10 down to .0575 W, and the size of the accumulator SRAM by a factor of 5 down to 58,276.8  $\mu\text{m}^2$ . This would bring the total power consumption down to 0.6225 W, a reduction of 45.4%, and area down to 300,333.709  $\mu\text{m}^2$ , for a reduction of 43.7%.

The synthesis clock period target was 1650ps, for a frequency of  $\sim 606$  MHz. The current design synthesizes successfully within this constraint, with a lowest critical path slack of 0ps. The critical path in this design is the data path from the accumulator SRAM, through the bias addition, to the output AXI bus. Timing constraints on this critical path could be relaxed by adding a pipeline register on the input of the bias addition operation, as address decoding takes approximately 400ps, so the path would be broken

up into a 400ps section and a 1200ps section. There are several reasons why this would not be a productive change.

First, the current 1650ps clock period was designed around the execution time of a single MAC operation in the systolic array. Reducing the critical path here would result in it being immediately replaced by one of the systolic array MACs as the critical path, which are irreducible. Second, the current clock period is designed so that the data bandwidth of the AXI bus matches the data bandwidth of the DRAM. Pipelining to increase clock speed has no benefit, since any gains would result in DRAM access speed becoming the bottleneck in my design.

## Benchmark Results

All benchmarks were run on a single complete forward and backward training pass over GPT-2 small (12 transformer layers,  $C=768$ , 12 attention heads) with batch size  $B=4$  and sequence length  $T=64$ . CPU timing uses the original software implementation with a warmup step, as typically the first step performs significantly worse than subsequent steps. Energy consumption information is acquired via the Linux RAPL sysfs interface, with readings taken immediately before and after each matmul call and accumulated across the full training step.

Accelerator timing is derived from a 64-bit cycle counter in the Verilator simulation that increments every simulated clock edge. The hardware abstraction layer (HAL) accumulates forward and backward cycle totals separately. Compute time is obtained by dividing the total simulated cycle count by the synthesized clock frequency of 606 MHz. Though the entire accelerated training layer is executed, execution time for the non-accelerated CPU work is taken directly from the software backend run and added to the projected hardware acceleration time. This is done because Verilator execution evicts cache data used by the CPU, which results in a much slower execution time for the CPU portions of the accelerated version, which is a function of the simulation method, rather than an actual behavior of the accelerated version of the algorithm.

The results of benchmark testing are as follows:

| Metric                     | Software | Accelerated | Ratio |
|----------------------------|----------|-------------|-------|
| Matrix multiplication time | 4.5055 s | 0.1779 s    | 25.3× |
| Non-matmul CPU time        | 0.7376 s | 0.7376 s    | 1.0×  |
| Total time                 | 5.2431 s | 0.9155 s    | 5.73× |

The speedup for matrix multiplication is 25.3x, resulting in a total speedup of 5.73x. Matrix multiplication is reduced from 86% of execution time in the software implementation to 19.4% in the accelerated version. This implies significant diminishing returns to further matrix multiplication speedup in terms of improvements in total training time. A perfect accelerator with infinite speedup would only improve the full training speedup from 5.73x to 7.14x. While there is still room for improvement on this front, it may be more efficient to find methods to accelerate other portions of the algorithm, where a smaller improvement would have a greater overall impact.

As mentioned in the roofline analysis section, this implementation is currently memory bound, with two different arithmetic intensities for the two targeted kernels: 16.0 GFLOP/Byte and 25.5 GFLOP/Byte, for an average AI of 22.5 FLOP/Byte. This reduces the theoretical accelerator throughput to 875 GFLOP/s from a compute bound limit of 1.24 TFLOP/s. The final throughput of my design is 733

GFLOP/s which sits approximately 16% under the theoretical limit. This 16% loss compared to the ideal is due to start and stop signals consuming otherwise productive cycles, and the initial fill and final drain cycles for each full tile having nothing to productively overlap with.

Energy consumption comparisons between the CPU and accelerator implementations are based on a combination of sources. Accelerator power consumption is based on execution time and power consumption information from synthesis. CPU energy consumption was determined via Intel RAPL, measuring exclusively matrix multiplication portions of the algorithm. These measurements target CPU power usage, ignoring DRAM transfer power consumption. RAPL measurements summed to 90.04J for the full training step on the CPU. For the accelerator, the total energy consumption was determined by multiplying the measured execution time for all three matrix operations, .1779s, by the power consumption of the accelerator chip, 1.14W, for a total energy consumption value of .2028J. This represents a 444x compute energy consumption reduction between the CPU and accelerator versions of the kernel. It should be noted that neither energy consumption measurement considered DRAM or AXI transfer costs, which would be roughly equivalent for each, but would significantly reduce the total energy consumption reduction value. In fact, the structure of the accelerator means that accelerator DRAM traffic is slightly higher than CPU DRAM traffic, which would further erode this advantage.

## Issues and Development Processes

The primary architectural constraint stemmed from the ASAP7 library's lack of native memory primitives. Synthesizing all storage from flip-flops inflated area and power, preventing performance optimizations like a double-buffered accumulator or an activation cache. A commercial 7nm FinFET PDK with native SRAM compilers would eliminate this bottleneck.

My implementation has gone through a number of iterations, reflecting multiple design paths that ultimately proved ineffective or not feasible. My final design actually matches my initial design in the broad sense, they are both 32x32 systolic arrays, but my original design was built before I had a full understanding of systolic array processing, and thus was profoundly inefficient. I therefore opted to implement a vectored MAC implementation, first with 128 MACs and then with 512 when the projected speedup for the 128 MAC version proved anemic. The 512 vectored implementation ran into problems when I realized that the forward matrix multiplication dimensions did not divide evenly into 512, which meant that the accelerator would not be fully utilized for part of the multiplication. This led me back to a systolic design, where I implemented a 32x16, 16x16, and 32x32 implementation, eventually deciding on the 32x32 implementation to maximize throughput.

Additional problems occurred during the design, including many problems building a genus.tcl file that would function correctly with the 23.10 version of the software. The reason for transitioning to Genus was due to Openlane 2 not supporting ASAP7 natively, and the computer I was working with not having enough RAM to synthesize my design. This led me to migrate to a PSU lab server (Centos7) and to the Cadence tool chain.

A final round of improvements was conducted when I realized that my design was lagging badly behind the memory bound roofline. I determined that the reasons for this were computation stalls during the writeback phase due to a lack of double buffering of the accumulation SRAM, and lost cycles every weight tile due to weight loading time. I solved the first problem by double buffering the accumulator SRAM, and the second by loading weights during the drain time of the previous tile, effectively using the 32 cycles where activations were already not being streamed.



Potential future improvements would require a large-scale redesign of the accelerator. The most promising potential avenue for improvement would be to shift the arithmetic intensity of the two kernels I am accelerating right, moving out of the memory bound region back to the compute bound. This would require a greater degree of data reuse than is currently the case in my design, which would in turn require more local SRAM. I estimate that approximately 5MB of SRAM would be required to effectively leverage data reuse in my design, which would in turn require a shift away from ASAP7 as my PDK. It would also require a significant controller redesign to manage data re-access. Shifting back into the compute bound region would allow weights to be loaded from SRAM during computation, which in turn would allow double buffering of the internal systolic array weight registers. This would allow continuous streaming of activations, avoiding fill/drain cycles and lifting my throughput to the compute bound limit.

## Figures

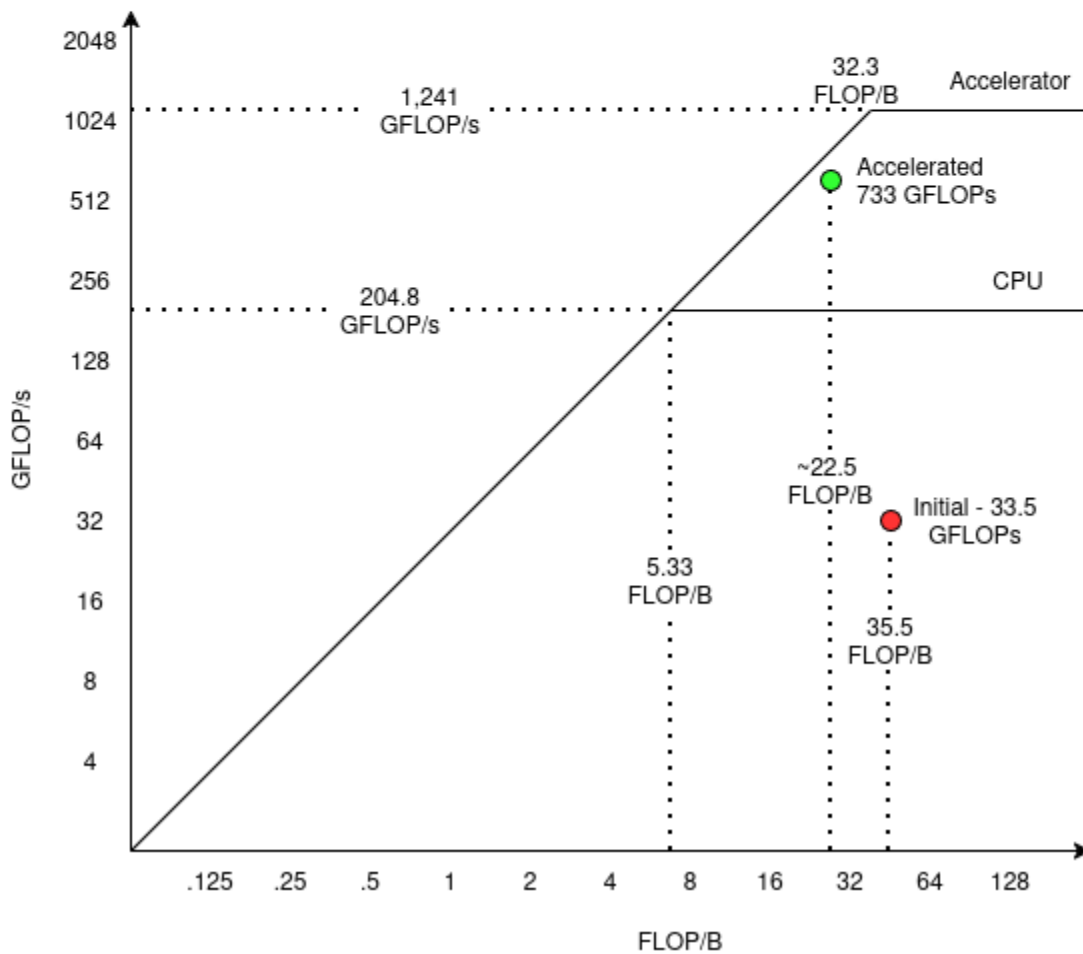


Figure 1.

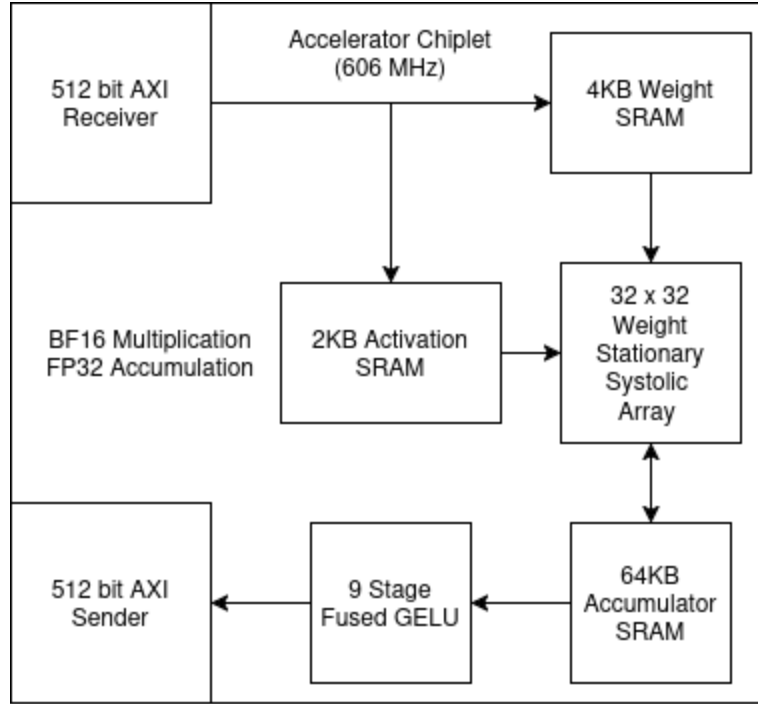


Figure 2.

## Acknowledgments

1. Karpathy, A. (2024). *llm.c: LLM training in simple, pure C/CUDA* (Version/Commit Hash) [Computer software]. GitHub. <https://github.com/karpathy/llm.c>
2. Clark, L. T., Vashishtha, V., Shifren, L., Gujja, A., Sinha, S., Cline, B., Ramamurthya, C., & Yeric, G. (2016). ASAP7: A 7-nm FinFET predictive process design kit. *Microelectronics Journal*, 53, 105-115.